

AFRILANG Stdlib

std/c/nlpcompress

Français. Bibliothèque AFRILANG 'std/c/nlpcompress' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : compLen, compUpper, compLower, compTrim, compSplit, compJoin, compReplace.

'import "std/c/nlpcompress"' · 'use nlpcompress'

- 'compLen(s text) → number' - 'compUpper(s text) → text' - 'compLower(s text) → text' - 'compTrim(s text) → text' - 'compSplit(s text, delim text) → list text' - 'compJoin(parts list text, delim text) → text' - 'compReplace(s text, from text, to text) → text'

English. AFRILANG library 'std/c/nlpcompress' (tier complex, category complex). Exports 7 function(s): compLen, compUpper, compLower, compTrim, compSplit, compJoin, compReplace.

'import "std/c/nlpcompress"' · 'use nlpcompress'

- 'compLen(s text) → number' - 'compUpper(s text) → text' - 'compLower(s text) → text' - 'compTrim(s text) → text' - 'compSplit(s text, delim text) → list text' - 'compJoin(parts list text, delim text) → text' - 'compReplace(s text, from text, to text) → text'

Catégorie / Category	Modules complexe (c/)
Niveau / Tier	complexe
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlpcompress' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : compLen, compUpper, compLower, compTrim, compSplit, compJoin, compReplace.

```
'import "std/c/nlpcompress"' · 'use nlpcompress'
```

```
- `compLen(s text) → number`
- `compUpper(s text) → text`
- `compLower(s text) → text`
- `compTrim(s text) → text`
- `compSplit(s text, delim text) → list text`
- `compJoin(parts list text, delim text) → text`
- `compReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlpcompress"
use nlpcompress
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- compLen(s text) → number•
compUpper(s text) → text•
compLower(s text) → text•
compTrim(s text) → text•
compSplit(s text, delim text) → list•
compJoin(parts list text, delim text) → text•
compReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/nlpcompress"
use nlpcompress
```

```
create result = compLen(/* args */)
say result
```

```
create result = compUpper(/* args */)
say result
```

```
create result = compLower(/* args */)
say result
```

```
create result = compTrim(/* args */)
say result
```

```
create result = compSplit(/* args */)
say result
```

```
create result = compJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlpcompress"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module nlpcompress

```
function compLen(s text) returns number
end
```

```
function compUpper(s text) returns text
end
```

```
function compLower(s text) returns text
end
```

```
function compTrim(s text) returns text
end
```

```
function compSplit(s text, delim text) returns list text
end
```

```
function compJoin(parts list text, delim text) returns text
end
```

```
function compReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/nlpcompress' (tier complex, category complex). Exports 7 function(s): compLen, compUpper, compLower, compTrim, compSplit, compJoin, compReplace.

```
'import "std/c/nlpcompress"' · 'use nlpcompress'
```

```
- `compLen(s text) → number`  
- `compUpper(s text) → text`  
- `compLower(s text) → text`  
- `compTrim(s text) → text`  
- `compSplit(s text, delim text) → list text`  
- `compJoin(parts list text, delim text) → text`  
- `compReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlpcompress"  
use nlpcompress
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- compLen(s text) → number•
compUpper(s text) → text•
compLower(s text) → text•
compTrim(s text) → text•
compSplit(s text, delim text) → list•
compJoin(parts list text, delim text) → text•
compReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/nlpcompress"  
use nlpcompress
```

```
create result = compLen(/* args */)   
say result
```

```
create result = compUpper(/* args */)   
say result
```

```
create result = compLower(/* args */)   
say result
```

```
create result = compTrim(/* args */)   
say result
```

```
create result = compSplit(/* args */)
say result
```

```
create result = compJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/nlpcompress"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlpcompress

```
function compLen(s text) returns number
end
```

```
function compUpper(s text) returns text
end
```

```
function compLower(s text) returns text
end
```

```
function compTrim(s text) returns text
end
```

```
function compSplit(s text, delim text) returns list text
end
```

```
function compJoin(parts list text, delim text) returns text
end
```

```
function compReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlpcompress"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/nlpcompress/>

Source (excerpt)

```
module nlpcompress

function compLen(s text) returns number
end

function compUpper(s text) returns text
end

function compLower(s text) returns text
end

function compTrim(s text) returns text
end

function compSplit(s text, delim text) returns list text
end

function compJoin(parts list text, delim text) returns text
end

function compReplace(s text, from text, to text) returns text
end
end
```